



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

محرك المعالجة عالي الأداء لنظام تجارة إلكتروني

High-Performance E-Commerce Backend Engine

تقرير مشروع مادة البرمجة المتوازية

تسنيم شادي الحكيم

آية عدنان عمران

مروة محمد زيواني

سارة عدنان عطية

إيمان نديم ياغي

مقدمة

لم تعد تطبيقات التجارة الإلكترونية مجرد واجهات لعرض المنتجات، بل أصبحت منصات حيوية تتعامل مع ملايين العمليات المتزامنة، وإن قدرة هذه الأنظمة على توفير تجربة مستخدم سلسة وفورية تعد عاملاً مهماً لنجاح المنتج التجاري. وهنا تبرز الفجوة بين التطبيقات التقليدية وتلك القادرة على استغلال قدرات المعالجات والخوادم الموزعة.

هذا المشروع لا يهدف فقط إلى بناء متجر إلكتروني، بل إلى تصميم وتنفيذ هيكلية برمجية تعالج بعمق تحديات الأداء باستخدام أساليب البرمجة المتوازية.

سيركز هذا التقرير على عرض الحلول التي تم التوصل إليها مع التفسير المنهجي لسبب اختيارها مع طريقة تطبيقها ونتائج أدائها، و سنقوم بتعريف مجموعة من المتطلبات الوظيفية الأساسية كحالة دراسية، تشمل إدارة المنتجات، وعربة التسوق، وعملية الدفع، والتي ستكون بمثابة الأرضية التي نختبر ونثبت عليها نماذج التوازي والتوزيع المقترحة.

المتطلبات الوظيفية المطبقة

انشاء مستخدمين

اضافة – سحب رصيد الى المحفظة

انشاء – تعديل – عرض – حذف المنتجات

اضافة كمية للمخزون

اضافة منتج للعربة وتعديل كميته وحذفه

تاكيد الطلب واجراء الدفع

تعديل حالة الطلب

امكانية الغاء الطلب اذا كان في مرحلة الانتظار

المتطلبات غير الوظيفية المطبقة

المتطلب الأول Concurrent Access & Data Integrity

في أنظمة التجارة الإلكترونية متعددة المستخدمين، قد يحاول أكثر من مستخدم تنفيذ عمليات شراء على نفس المنتج في نفس الوقت، مما يؤدي إلى ظهور مشكلات تتعلق بتزامن العمليات (Concurrency Problems). ومن أبرز هذه المشكلات مشكلة قراءة البيانات قبل تحديثها (Dirty Read / Race Condition)، حيث يتم التحقق من كمية المنتج المتوفرة قبل أن يتم تحديث الكمية فعلياً نتيجة عملية شراء أخرى.

في نظام المتجر الإلكتروني، عند قيام المستخدم بإتمام عملية الدفع، يقوم النظام أولاً بالتحقق من توفر الكمية المطلوبة من المنتج.

لكن في حالة وجود عدة مستخدمين يحاولون شراء نفس المنتج في نفس اللحظة، قد تحدث المشكلة التالية:

- المستخدم الأول يطلب شراء آخر قطعة من المنتج .
- قبل أن يتم تحديث الكمية في قاعدة البيانات، يقوم المستخدم الثاني بإرسال طلب شراء لنفس المنتج .
- النظام يقرأ الكمية القديمة لكلا المستخدمين ويعتبر أن المنتج ما يزال متوفرًا .
- يتم قبول الطلبين معًا رغم أن الكمية الحقيقية لا تسمح بذلك .

وهذا يؤدي إلى:

- بيع منتجات غير متوفرة .
- حدوث أخطاء في المخزون .
- فقدان موثوقية النظام .
- تعارض البيانات داخل قاعدة البيانات.

الحلول المطبقة لمعالجة المشكلة

1- تنفيذ عملية التحقق والشراء داخل Transaction واحدة

تم استخدام مفهوم **Database Transaction** بحيث تصبح جميع خطوات عملية الشراء عملية واحدة مترابطة .

آلية العمل:

تشمل العملية الخطوات التالية داخل Transaction واحدة:

1. التحقق من توفر الكمية .
2. خصم الكمية من المخزون .
3. إنشاء سجل الطلب .
4. تحديث حالة الدفع .
5. حفظ جميع التغييرات .

وفي حال فشل أي خطوة من الخطوات السابقة:

- يتم إلغاء جميع التعديلات السابقة تلقائيًا (Rollback)
- تعود قاعدة البيانات إلى حالتها الأصلية قبل بدء العملية.

أما إذا نجحت جميع الخطوات: يتم تثبيت التعديلات.

أهمية هذا الحل:

هذا الأسلوب يحقق خاصية Atomicity من خصائص ACID في قواعد البيانات، والتي تعني أن العملية إما تنجح بالكامل، أو تفشل بالكامل دون ترك بيانات غير مكتملة.

2- استخدام SELECT FOR UPDATE

عند قيام مستخدم بقراءة بيانات منتج باستخدام FOR UPDATE يتم قفل الصف (Row Lock) الخاص بالمنتج داخل قاعدة البيانات واي مستخدم آخر يحاول تعديل نفس المنتج يجب أن ينتظر حتى انتهاء العملية الحالية، ويحقق:

- منع قراءة بيانات قديمة قبل تحديثها .
- ضمان تنفيذ العمليات بالتسلسل الصحيح .
- منع تضارب تحديثات المخزون .
- الحفاظ على دقة بيانات المنتجات.

The screenshot shows the Django administration interface. The top navigation bar includes 'Django administration' and user information 'WELCOME, EMAN. VIEW SITE / CHANGE PASSWORD / LOG OUT'. The breadcrumb trail is 'Home > Store > Products > iPhone'. The left sidebar contains a search bar and a list of menu items: 'AUTHENTICATION AND AUTHORIZATION' (Groups, Users), 'PERIODIC TASKS' (Clocked, Crontabs, Intervals, Periodic tasks, Solar events), and 'STORE' (Banks, Cart products). The main content area is titled 'Change product' and shows the 'iPhone' product form. The form fields are: 'Name:' (iPhone), 'Price:' (100.00), and 'Quantity:' (1). At the bottom of the form are four buttons: 'SAVE', 'Save and add another', 'Save and continue editing', and 'Delete'.

The screenshot shows a user interface with a dark background. It features a 'Cart:' section with a dropdown menu showing 'Aya Cart' and three icons: a pencil, a plus sign, and an eye. Below this is a 'Product:' section with a dropdown menu showing 'iPhone' and the same three icons. At the bottom is a 'Quantity:' section with a text input field containing the number '1'.

Cart:

marwa Cart ▾



Product:

iPhone ▾



Quantity:

1

```
(venv) D:\ecommerce\PP-Ecommerce-Project>python test_concurrency.py
```

marwa

Status: 400

Response: {"message":"Cart is empty"}

Aya

Status: 200

Response: {"message":"Payment successful","order_id":1}

```
(venv) D:\ecommerce\PP-Ecommerce-Project>
```

النتيجة النهائية

تم ضمان سلامة البيانات ومنع تضارب العمليات المتزامنة، وموثوقية عمليات الدفع والشراء داخل النظام، كما ساهم هذا الحل في جعل النظام قادرا على التعامل مع عدد كبير من المستخدمين في نفس الوقت دون حدوث أخطاء في بيانات المنتجات أو الطلبات.

المتطلب الثاني Resource Management & Capacity

عملية إدارة الموارد تُعنى بالاستهلاك المناسب لموارد الحاسوب من أجل تنفيذ عدد من العمليات على التوازي دون حدوث تضارب ودون حدوث بطء بالتنفيذ

طرق التنفيذ:

هناك عدة طرق ممكنة لإدارة هذه الموارد وفعليا لا يتم اختيار أحد هذه الطرق تفضيلا عن الأخرى وإنما يتم التكامل بينها جميعا لتعطي النتيجة النهائية, حيث يتم اختيار ما يتم تطبيقه من هذه الطرق بحسب نتيجة اختبارات لاستهلاك الموارد وليس بشكل اعتباطي, ومن هذه الطرق: thread pooling, connection pooling, queue, async queues, batch processing

في هذا الطلب تم تطبيق مفاهيم ال connection pooling, thread pooling and queue

بينما async queues and batch processing فقد تم تطبيقهم بالطلب الثالث والرابع

سبب اختيار تطبيق هذه المفاهيم:

بحسب نتائج الاختبار, يتضح أن موارد الجهاز يتم استهلاكها بشكل كبير ويتم إنشاء مئات threads ومع ذلك يفشل النظام في تلبية جميع الطلبات, لذلك تطبيق إدارة ال threads وإدارة الاتصال مع قاعدة البيانات تساعد كثيرا على الاستفادة الصحيحة من الموارد في هذه الحالة

شرح المفاهيم المطبقة:

Thread pool -1

ما تم تطبيقه هو شيء مشابه لمفاهيم ال thread pool عن طريق استخدام سيرفر waitress الذي يقدم خدمة multithreading بشكل داخلي ويحتوي على queue لتنظيم ال requests عندما تمتلئ ال threads عند دراسة نظام ال backend تبين أن معظم العمليات هي I/O bound وأغلبها تتعلق بالوصول لقاعدة المعطيات, لهذا السبب تم اعتماد تحديد عدد ال:

- Threads عن طريق المعادلة: $threads = cpu_cores * 2$
أما الاستهلاك من ناحية ال cpu فهو قليل لذلك لا حاجة ل workers إضافية
- تحديد عدد ال connection ، وهو عدد الاتصالات المسموح بها بشكل متزامن مع السيرفر في نفس اللحظة, سواء كانت Requests قيد المعالجة أو Requests تنتظر ضمن ال queue.
تم تحديده باستخدام المعادلة: $connections = threads * 50$
وذلك للسماح باستقبال عدد كبير من الاتصالات دون رفضها مباشرة عند امتلاء ال threads, حيث يتم وضع الطلبات الزائدة ضمن ال queue إلى حين توفر thread فارغ لمعالجتها.
اختيار قيمة أكبر من عدد ال threads مهم في الأنظمة ال I/O bound لأن أغلب ال threads تقضي وقتها بانتظار

عمليات قاعدة البيانات ولا تكون مستهلكة للـ CPU بشكل فعلي, وبالتالي يمكن استقبال عدد اتصالات أكبر من عدد الـ threads نفسه دون التأثير الكبير على المعالج.

-2 connection pooling

قمنا بتطبيق المفهوم عن طريق تحديد المدة التي يبقى فيها الاتصال مفتوحاً مع قاعدة البيانات, حيث عملية إغلاق الاتصال وإنشاءه مرة ديدة لكل request مع الـ database يستغرق وقت وموارد أيضاً, وبما أننا نحاول استهلاك الموارد بشكل متوازن لتحسين الإنتاجية, لجأنا لهذا الحل كي لا يستهلك الاتصال بقاعدة البيانات بحد ذاته موارد ووقت

الاختبارات:

تم الاختبار على الـ create_order api (اسمه في الـ views هو cart_confirm_payment)

وتم اختبار 300 طلب بمعدل 10 طلبات كل ثانية

قبل التحسين:

لا يوجد connection pooling ولا يتم إدارة الـ threads



LOCUST

Host

http://127.0.0.1:8000/

Status

RUNNING

Users

300

RPS

45.13

Failures

4%

EDIT

LOADING

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	//api/cart/confirm/1	9	0	200	6800	6800	1785.81	56	6752	27	0.2	0
POST	//api/cart/confirm/10	1	1	11256.58	11000	11000	11256.58	11257	11257	228158	0.1	0.1
POST	//api/cart/confirm/100	1	1	13209.61	13000	13000	13209.61	13210	13210	228183	0.1	0.1
POST	//api/cart/confirm/102	25	0	140	6600	7700	1180.23	12	7657	27	0.4	0
POST	//api/cart/confirm/107	8	0	130	7100	7100	1919.36	11	7065	27	0.1	0
POST	//api/cart/confirm/109	1	1	16037	16000	16000	16037	16037	16037	228183	0.1	0.1
POST	//api/cart/confirm/11	10	0	140	8000	8000	1494.37	18	8035	27	0.1	0
POST	//api/cart/confirm/111	21	0	240	6500	8300	1507.79	12	8276	27	0.4	0
POST	//api/cart/confirm/113	16	1	110	8600	8600	1303.75	11	8591	14286.75	0.2	0
POST	//api/cart/confirm/115	25	0	180	6400	7300	1245.95	9	7280	27	0.4	0
POST	//api/cart/confirm/12	1	1	18828.95	19000	19000	18828.95	18829	18829	228158	0	0
POST	//api/cart/confirm/120	38	0	120	7100	8200	1251.02	9	8180	27	0.5	0
POST	//api/cart/confirm/122	9	1	1400	15000	15000	3370.88	17	15025	25377.67	0.3	0.1
POST	//api/cart/confirm/123	1	1	16972.13	17000	17000	16972.13	16972	16972	228183	0.1	0.1

STATISTICS	CHARTS	FAILURES	EXCEPTIONS	CURRENT RATIO	DOWNLOAD DATA	LOGS
# Failures	Method	Name	Message	First Seen	Last Seen	
1	POST	//api/cart/confirm/10	HTTPError(500 Server Error: Internal Server Error for url: //api/cart/confirm/10)	5/18/2026, 10:39:00 PM	5/18/2026, 10:39:00 PM	
1	POST	//api/cart/confirm/100	HTTPError(500 Server Error: Internal Server Error for url: //api/cart/confirm/100)	5/18/2026, 10:39:02 PM	5/18/2026, 10:39:02 PM	
1	POST	//api/cart/confirm/109	HTTPError(500 Server Error: Internal Server Error for url: //api/cart/confirm/109)	5/18/2026, 10:39:06 PM	5/18/2026, 10:39:06 PM	
1	POST	//api/cart/confirm/113	HTTPError(500 Server Error: Internal Server Error for url: //api/cart/confirm/113)	5/18/2026, 10:38:52 PM	5/18/2026, 10:38:52 PM	
1	POST	//api/cart/confirm/12	HTTPError(500 Server Error: Internal Server Error for url: //api/cart/confirm/12)	5/18/2026, 10:39:12 PM	5/18/2026, 10:39:12 PM	



الاختبار يوضح مايلي:

الوقت الذي يستغرقه لمعالجة الطلب يصل في الكثير من الحالات لـ 20 ثانية، وحصل فشل في عدة طلبات بسبب خطأ داخلي بالسيرفر نتيجة الضغط إضافة إلى أن معدل عدد الطلبات في الثانية RPS : تقريبا 45



أما بالنسبة لاستهلاك الموارد فتوضح المخططات أن الاستهلاك عالي جدا حيث يستهلك تقريبا 90% من ال CPU وقام بإنشاء ما يقارب 300 thread ورغم ذلك كان هناك بعض الطلبات التي تفشل

بعد إجراء التحسينات:

LOCUST												Host http://127.0.0.1:8000/	Status STOPPED	RPS 136.87	Failures 0%	NEW	RESET	⚙
STATISTICS																		
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s						
POST	//api/cart/confirm/1	65	0	91	220	290	95.63	9	288	27	1.3	0						
POST	//api/cart/confirm/100	65	0	92	200	260	96.36	9	256	27	1.2	0						
POST	//api/cart/confirm/102	30	0	130	290	350	135.02	9	345	27	0.7	0						
POST	//api/cart/confirm/103	28	0	120	220	270	120.15	37	270	27	0.6	0						
POST	//api/cart/confirm/104	23	0	140	330	330	137.99	14	334	27	0.6	0						
POST	//api/cart/confirm/105	62	0	160	320	380	162.29	11	376	27	1.7	0						
POST	//api/cart/confirm/106	58	0	180	290	410	183.2	15	407	27	1.8	0						
POST	//api/cart/confirm/107	32	0	90	230	290	110.02	10	286	27	0.6	0						
POST	//api/cart/confirm/109	27	0	140	210	250	121.2	15	248	27	0.6	0						
POST	//api/cart/confirm/11	61	0	97	240	350	103.91	11	350	27	1.4	0						
POST	//api/cart/confirm/110	22	0	150	240	350	151.85	37	345	27	0.6	0						
POST	//api/cart/confirm/111	26	0	140	220	220	136.45	10	217	27	0.6	0						
POST	//api/cart/confirm/112	29	0	110	280	310	114.27	12	309	27	0.6	0						
POST	//api/cart/confirm/113	56	0	130	280	360	122.46	13	364	27	1.2	0						

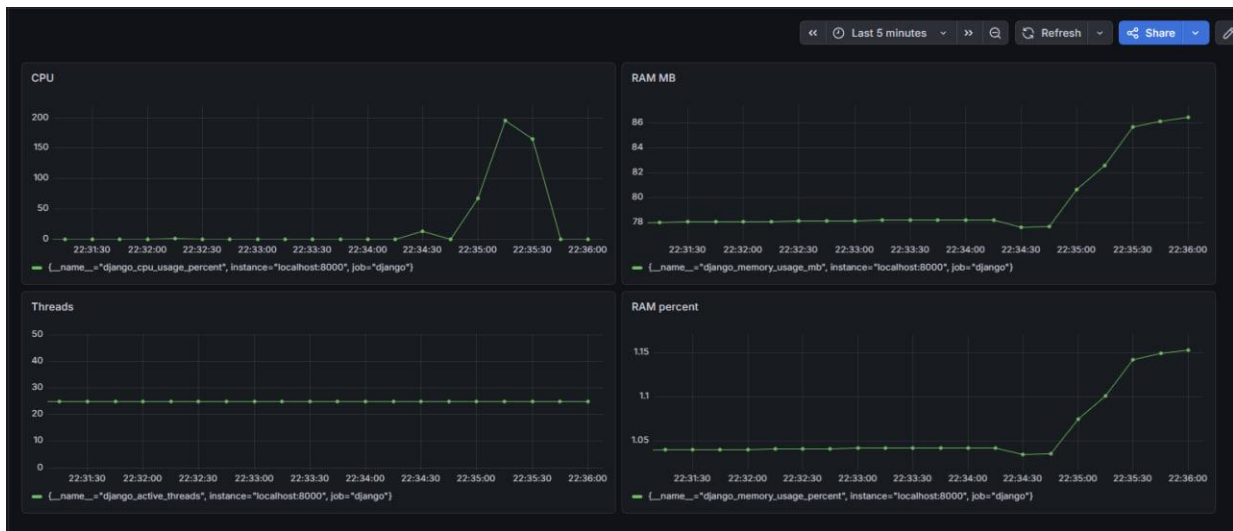


```
(PP-Ecommerce-Project) PS C:\Users\Tsunami\Desktop\PP-Ecommerce-Project> python run_server.py

=====
CPU cores: 12
Threads: 24
Connection limit: 1200
=====

INFO:waitress:erving on http://127.0.0.1:8000
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
```

نلاحظ تحسن كبير بزمان الاستجابة (وسطيا ms400) ولا يوجد أي أخطاء أو فشل اتصال، والصورة الأخيرة توضح أن ال queue لا تحتوي إلا على request واحد تقريبا دائما مما يعني أن عدد الطلبات التي تنتظر لتتم معالجتها قليل



نلاحظ أن استهلاك الموارد أصبح متوازن وطبيعي، ورغم أن الاستهلاك هنا أقل إلى أنه بسبب الإدارة الصحيحة وتنظيم ال thread كانت الإنتاجية أفضل

المتطلب الثالث Asynchronous Queues

في الأنظمة الحديثة، تُعدّ مسألة تحسين الأداء وتقليل زمن الاستجابة من أهم التحديات، خاصة عند التعامل مع عمليات ثقيلة مثل توليد الملفات أو إرسال البريد الإلكتروني. في هذا المشروع، ظهرت الحاجة إلى معالجة هذه المشكلة من خلال الانتقال من التنفيذ المتزامن (Synchronous) إلى التنفيذ غير المتزامن.

المشكلة قبل تطبيق الحل:

في النسخة الأولية من النظام، كانت عملية تأكيد الطلب (Order Confirmation) تتضمن تنفيذ عدة مهام داخل نفس الطلب (HTTP Request)، وهي:

- التحقق من الدفع
- إنشاء الطلب

وعند نجاح الطلب يتم :

- توليد فاتورة بصيغة PDF
- إرسال الفاتورة عبر البريد الإلكتروني

وكانت هذه العمليات تنفذ بشكل متسلسل ومتزامن، مما أدى إلى:

- زيادة زمن الاستجابة بشكل ملحوظ، حيث كان المستخدم ينتظر حتى انتهاء جميع العمليات.
- تعطيل (Blocking) الخيط التنفيذي (Thread) الخاص بالطلب، مما يقلل من قدرة النظام على معالجة طلبات أخرى.
- قابلية عالية للفشل، إذ إن فشل أي جزء (مثل إرسال البريد) يؤدي إلى فشل العملية كاملة.
- تجربة مستخدم غير جيدة نتيجة التأخير.

الهدف من الحل:

- فصل العمليات الثقيلة عن الطلب الأساسي
- تنفيذ هذه العمليات في الخلفية
- تقليل زمن الاستجابة إلى الحد الأدنى
- تحسين اعتمادية النظام (Reliability)

الحل المعتمد:

تم اعتماد البنية التالية:

- استخدام Celery كإطار لتنفيذ المهام غير المتزامنة

- استخدام Redis كوسيط لتخزين المهام

آلية العمل:

- 1- يقوم المستخدم بإرسال طلب تأكيد الشراء عبر الواجهة (API Request).
 - 2- عند استقبال الطلب، يقوم النظام أولاً بمحاكاة مرحلة معالجة أولية (Processing Delay) لتمثيل الزمن الذي تستغرقه الأنظمة عادة في التحقق من البيانات، وذلك باستخدام تأخير زمني بسيط.
 - 3- بعد ذلك، يتم تنفيذ محاكاة لعملية الدفع (Payment Simulation)، حيث:
 - يتم إدخال تأخير إضافي يمثل زمن التواصل مع النظام البنكي.
 - يتم تحديد نتيجة الدفع (نجاح أو فشل) بشكل عشوائي لمحاكاة سلوك الأنظمة الواقعية.
 - 4- في حال نجاح عملية الدفع (Payment Success):
 - يتم إنشاء الطلب (Order) وتحديث حالة النظام.
 - عند نجاح الطلب يتم إرجاع الاستجابة للمستخدم مباشرة دون انتظار العمليات اللاحقة.
 - 5- بالتوازي، يقوم النظام بإرسال مهمة (Task) إلى Celery عبر Redis Broker، دون إيقاف أو تأخير استجابة المستخدم.
 - 6- يقوم Celery Worker بالتقاط المهمة وتنفيذها في الخلفية، وتشمل:
 - محاكاة معالجة طويلة نسبياً (Heavy Processing Delay)
 - توليد ملف الفاتورة (PDF Invoice)
 - إرسال بريد إلكتروني للمستخدم مرفقاً بالفاتورة
 - 7- في حال حدوث أي خطأ أثناء تنفيذ المهمة:
 - يتم إعادة المحاولة (Retry) بشكل تلقائي وفق سياسة محددة.
 - وفي حال فشل جميع المحاولات، يتم اعتبار المهمة فاشلة ومعالجتها لاحقاً.
- عند تصميم نظام معالجة مهام غير متزامنة (Asynchronous Processing)، كان لا بد من اختيار وسيط رسائل (Message Broker) مناسب لإدارة تنفيذ المهام الخلفية عبر Celery. وقد تم دراسة ثلاث خيارات رئيسية:

العيوب	المميزات	آلية العمل	
لا يدعم بشكل افتراضي: Dead Letter Queue و Advanced Routing	سرعة عالية جداً (Low Latency) سهل الإعداد (Lightweight) مناسب للأنظمة التي تحتاج أداء سريع (مثل إرسال الإيميلات)	يقوم Celery بوضع الرسائل (Tasks) ضمن Redis Queue	Redis

يقوم Worker بسحب الرسائل وتنفيذها يتم تخزين الرسائل في الذاكرة (RAM) مما يجعل الأداء سريع جداً		في حال انطفاء السيرفر ممكن فقدان بعض الرسائل (إذا لم يتم تفعيل persistence)
RabbitMQ الرسائل تُرسل إلى Exchange يتم توجيهها إلى Queue بناءً على Routing Rules Worker يستهلك الرسائل من Queue	دعم قوي جداً لـ Dead Letter Queues (DLQ) و Retry Policies و Message Acknowledgement وضمان أعلى لوصول الرسائل (Reliability) و مناسب للأنظمة الكبيرة (Enterprise Systems)	أعقد في الإعداد أبسط نسبياً من Redis بسبب الكتابة على Disk وإدارة الرسائل بشكل أكثر أماناً
Memory Broker هو وسيط مؤقت يعتمد على الذاكرة فقط داخل نفس العملية.	سريع جداً (لأنه لا يوجد Network) لا يحتاج أي إعداد خارجي	لا يعمل بين أكثر من Process لا يصلح للإنتاج الرسائل تضيق عند إيقاف البرنامج

التقنية المختارة:

تم اختيار Redis لأن النظام الحالي لا يحتاج تعقيد RabbitMQ فالمطلوب إرسال إيميلات وإنشاء PDF، كما ان الأداء السريع أهم من التعقيد، ولسهولة الإعداد والتشغيل.

قبل تطبيق الحل كان المستخدم ينتظر عند الدفع على الأقل 10 ثواني وفي الحياة الواقعية والردود البنكية قد تستغرق الموافقة أكثر

اما عند تطبيق الحل ينتظر المستخدم ثانيتين فقط لرد ال payment gateways وعند نجاح الطلب ينتظر المستخدم عملية انشاء ال pdf وارسال ال email والتي قد تستغرق الى 4 ثواني عدا حالات الفشل اما بعد تطبيق الحال فلا ينتظر ابدا.

معالجة الأخطاء:

تمت إضافة آلية لمعالجة الأخطاء داخل مهام Celery بهدف ضمان استمرارية النظام وعدم فقدان العمليات في حال حدوث أعطال غير متوقعة.

تعتمد هذه الآلية على إحاطة منطق المهمة ببنية try/except، بحيث يتم التقاط أي استثناء (Exception) قد يحدث أثناء تنفيذ المهمة، سواء كان ناتجاً عن:

- فشل في الاتصال (مثل خدمة البريد الإلكتروني)
- خطأ في توليد ملف الفاتورة
- مشاكل في الوصول إلى قاعدة البيانات

عند حدوث أي خطأ، لا يتم إنهاء المهمة مباشرة، وإنما يتم تفعيل آلية إعادة المحاولة (Retry) الخاصة بـ Celery، وذلك عبر استدعاء إعادة جدولة نفس المهمة للتنفيذ مرة أخرى لاحقاً.

وهذا يمنع فشل المهمة بشكل نهائي عند أول خطأ، ويزيد من موثوقية النظام (Reliability).

آلية إعادة المحاولة:

تم ضبط إعدادات إعادة المحاولة في المهمة كما يلي:

- الحد الأقصى لعدد المحاولات: 3 مرات
- الفاصل الزمني بين كل محاولة: 5 ثواني

أي أن النظام يعمل وفق السلوك التالي:

- 1- عند حدوث خطأ أثناء تنفيذ المهمة يتم إيقاف التنفيذ الحالي مؤقتاً.
- 2- يقوم Celery بإعادة إرسال المهمة إلى الـ Queue لتتم معالجتها مرة أخرى بعد 5 ثواني.
- 3- يتم تكرار هذه العملية حتى 3 محاولات كحد أقصى.
- 4- في حال استمرار الفشل بعد جميع المحاولات يتم اعتبار المهمة فاشلة بشكل نهائي.

المتطلب الرابع Batch Processing

في الطريقة التقليدية لعملية الجرد وإرسال التقارير تتم معالجة كل عملية بيع أو سجل بشكل منفصل وضمن مسار تنفيذ واحد فقط.

حيث يقوم النظام بقراءة جميع سجلات المبيعات واحداً تلو الآخر، ثم حساب الإجماليات، وتحديث البيانات، وإنشاء التقارير بشكل متتابع. وبعد الانتهاء من جميع العمليات يتم إرسال التقرير النهائي.

ومع زيادة حجم البيانات تصبح هذه الطريقة بطيئة وتستهلك موارد كبيرة، كما أن أي خطأ أو تأخير قد يؤثر على عملية الجرد بالكامل، لذلك فهي غير مناسبة لمعالجة البيانات الضخمة.

لحل مشكلة معالجة عدد كبير من الطلبات دفعة واحدة، تم استخدام مفهوم الـ Batch Processing، بحيث يتم تقسيم البيانات إلى أجزاء صغيرة (Chunks) بدلاً من معالجة جميع الطلبات دفعة واحدة.

بداية الحل Fixed Chunking:

في البداية تم تجربة أسلوب Fixed Chunking حيث يتم تقسيم البيانات إلى batches ثابتة الحجم.
مثال:

`chunk_size = 500`

طريقة العمل:

1. جلب أول 500 Order
2. حساب المبيعات الخاصة بهم
3. الانتقال إلى الـ 500 التالية
4. تكرار العملية حتى انتهاء جميع الطلبات

مشاكل Fixed Chunking:

رغم أن هذا الحل أفضل من معالجة جميع البيانات دفعة واحدة، لكنه كان يحتوي على مشكلة مهمة:

1. حجم الـ chunk ثابت دائماً
2. لا يتغير حسب الضغط أو سرعة التنفيذ
3. قد يكون كبير جداً فيسبب ضغط على النظام أو صغير جداً فيسبب عدد كبير من الـ queries وبالتالي بطء غير ضروري .

أي أن النظام لم يكن يتكيف مع حالة السيرفر أو حجم البيانات.

```
0-ecf20005d059] Received
[2026-05-18 17:53:00,050: WARNING/MainProcess] =====
[2026-05-18 17:53:00,051: WARNING/MainProcess] Current chunk size: 100
[2026-05-18 17:53:00,052: WARNING/MainProcess] Execution time: 0.0099 sec
[2026-05-18 17:53:00,053: WARNING/MainProcess] Orders processed: 100
[2026-05-18 17:53:00,054: WARNING/MainProcess] =====
=====
[2026-05-18 17:53:00,064: WARNING/MainProcess] =====
[2026-05-18 17:53:00,077: WARNING/MainProcess] Current chunk size: 100
[2026-05-18 17:53:00,081: WARNING/MainProcess] Execution time: 0.0244 sec
[2026-05-18 17:53:00,083: WARNING/MainProcess] Orders processed: 100
[2026-05-18 17:53:00,083: WARNING/MainProcess] =====
=====
[2026-05-18 17:53:00,089: WARNING/MainProcess] =====
[2026-05-18 17:53:00,089: WARNING/MainProcess] Current chunk size: 100
[2026-05-18 17:53:00,090: WARNING/MainProcess] Execution time: 0.0489 sec
[2026-05-18 17:53:00,091: WARNING/MainProcess] Orders processed: 100
[2026-05-18 17:53:00,091: WARNING/MainProcess] =====
=====
[2026-05-18 17:53:00,098: WARNING/MainProcess] =====
[2026-05-18 17:53:00,099: WARNING/MainProcess] Current chunk size: 100
[2026-05-18 17:53:00,099: WARNING/MainProcess] Execution time: 0.0574 sec
[2026-05-18 17:53:00,100: WARNING/MainProcess] Orders processed: 100
[2026-05-18 17:53:00,100: WARNING/MainProcess] =====
=====
[2026-05-18 17:53:00,105: WARNING/MainProcess] =====
[2026-05-18 17:53:00,105: WARNING/MainProcess] Current chunk size: 100
[2026-05-18 17:53:00,106: WARNING/MainProcess] Execution time: 0.0651 sec
[2026-05-18 17:53:00,106: WARNING/MainProcess] Orders processed: 100
[2026-05-18 17:53:00,106: WARNING/MainProcess] =====
```

تم الانتقال إلى Adaptive Chuncking

وهو أسلوب يقوم بتغيير حجم الـ chunk بشكل ديناميكي حسب سرعة التنفيذ.

طريقة العمل:

1. قياس زمن تنفيذ الـ batch الحالية

$execution_time = time.time() - start_time$

2. مقارنة الزمن مع Target Time محدد مسبقاً

$target_time = 0.2$

3. حساب الحجم الجديد للـ chunk

$time_ratio = target_time / execution_time$

4. تكبير أو تصغير حجم الـ chunk حسب الأداء، فإذا كان التنفيذ سريع يتم زيادة حجم الـ chunk أما إذا كان التنفيذ بطيئاً يتم تقليل حجم الـ chunk

تأثير الحل على الأداء:

1. تحسين الأداء
2. تحسين قابلية التوسع
3. تقليل استهلاك الذاكرة
4. تقليل الضغط على قاعدة البيانات

```
[2026-05-18 17:43:00,079: WARNING/MainProcess] =====
====
[2026-05-18 17:43:00,080: WARNING/MainProcess] Current chunk size: 500
[2026-05-18 17:43:00,081: WARNING/MainProcess] Execution time: 0.0360 sec
[2026-05-18 17:43:00,082: WARNING/MainProcess] Orders processed: 500
[2026-05-18 17:43:00,083: WARNING/MainProcess] Next chunk size: 2780
[2026-05-18 17:43:00,083: WARNING/MainProcess] =====
====
[2026-05-18 17:43:00,104: WARNING/MainProcess] =====
====
[2026-05-18 17:43:00,105: WARNING/MainProcess] Current chunk size: 2780
[2026-05-18 17:43:00,106: WARNING/MainProcess] Execution time: 0.0208 sec
[2026-05-18 17:43:00,106: WARNING/MainProcess] Orders processed: 473
[2026-05-18 17:43:00,107: WARNING/MainProcess] Next chunk size: 5000
[2026-05-18 17:43:00,107: WARNING/MainProcess] =====
```

تصحيح الأخطاء:

1. تم إضافة Error Handling داخل الـ Celery Task باستخدام

```
try:
    .....
except Exception as exc:
    raise self.retry(exc=exc)
```

في حال حدوث أي Exception غير متوقعة يقوم Celery بإعادة المحاولة تلقائياً بدلاً من فشل المهمة بالكامل.

2. Retry

max_retries = 3

default_retry_delay = 5

أي إعادة المحاولة 3 مرات كل 5 ثواني بين المحاولات

:Idempotency

تشغيل المهمة أكثر من مرة لن يؤدي إلى نتائج خاطئة أو مضاعفة البيانات

DailySummary.objects.update_or_create()

بدل create ، بذلك لن يتم إنشاء duplicate rows عند تشغيل التاسك أكثر من مرة حيث سيتم تحديث نفس السجل الخاص بنفس التاريخ وهكذا يمنع التكرار وتضارب البيانات.

قبل تطبيق الحلول:

```
[2026-05-18 20:14:38,489: INFO/MainProcess] mingle: searching for neighbors
[2026-05-18 20:14:39,550: INFO/MainProcess] mingle: all alone
[2026-05-18 20:14:39,638: INFO/MainProcess] celery@Sara ready.
[2026-05-18 20:15:00,005: INFO/MainProcess] Task store.tasks.run_fixed_sales_job[d26cb10e-dfe0-49e5-be0d-69670591bba0] received
[2026-05-18 20:15:26,408: WARNING/MainProcess] =====
[2026-05-18 20:15:26,408: WARNING/MainProcess] Execution time: 26.3804 sec
[2026-05-18 20:15:26,409: WARNING/MainProcess] Orders processed: 972027
[2026-05-18 20:15:26,409: WARNING/MainProcess] =====
=====
[2026-05-18 20:15:26,423: INFO/MainProcess] Task store.tasks.run_fixed_sales_job[d26cb10e-dfe0-49e5-be0d-69670591bba0] received
```

في الأنظمة الحديثة، خصوصاً أنظمة التجارة الإلكترونية، يزداد عدد الطلبات الواردة إلى النظام بشكل كبير عند وجود عدد كبير من المستخدمين المتصلين في نفس الوقت. ولضمان استقرار النظام وسرعة الاستجابة، يتم استخدام تقنيات توزيع الأحمال (Load Distribution) لتوزيع الطلبات على عدة خوادم بدلاً من الاعتماد على خادم واحد فقط.

يهدف هذا المتطلب إلى محاكاة طريقة عمل أنظمة توزيع الأحمال المستخدمة في الأنظمة الحقيقية، وذلك من خلال توزيع طلبات المستخدمين على أكثر من خادم باستخدام أكثر من استراتيجية توزيع.

المشكلة:

في حال اعتماد النظام على خادم واحد فقط:

- قد يتعرض الخادم للضغط الزائد .
- ترتفع مدة الاستجابة .
- قد يتوقف النظام عند زيادة عدد المستخدمين .
- تنخفض كفاءة الأداء والاستقرار .

لذلك تم تصميم النظام بحيث:

- يتم توزيع الطلبات على عدة خوادم (Servers).
- يتم منع تكديس الطلبات على خادم واحد .
- يتم استغلال موارد النظام بشكل أفضل.

الاستراتيجيات المستخدمة في النظام:

تم تطبيق نموذج Hybrid Load Balancing والذي يجمع بين Round Robin Load Balancing و Smart Load Balancing.

وذلك لمحاكاة الأنظمة الحديثة التي تستخدم أكثر من أسلوب توزيع للحصول على أفضل أداء ممكن.

1- Round Robin Load Balancing :

تعد خوارزمية Round Robin من أبسط وأكثر طرق توزيع الأحمال شيوعاً. تعتمد هذه الطريقة على توزيع الطلبات بشكل دوري ومتسلسل على الخوادم المتوفرة.

تعد هذه الطريقة:

- بسيطة وسهلة التنفيذ .
- تحقق توزيعاً أولياً متوازناً .
- تقلل من احتمالية تكديس الطلبات على خادم واحد .
- مناسبة للأنظمة التي تكون فيها قدرات الخوادم متقاربة.

إلا أن هذه الطريقة لا تأخذ بعين الاعتبار:

- الضغط الحالي على الخادم .
- عدد العمليات الجارية .
- استهلاك الذاكرة أو المعالج .

فقد يتم إرسال طلب جديد إلى خادم مشغول بالفعل، بينما يوجد خادم آخر أقل ضغطاً، ولهذا السبب تم إضافة أسلوب أكثر آخر داخل النظام.

-2 Smart Load Balancing :

يقوم النظام بمحاكاة أسلوب Smart Load Balancing بحيث لا يتم الاعتماد فقط على التوزيع الدوري، وإنما يتم أيضاً تحليل حالة الخوادم واختيار الخادم الأقل ضغطاً.

يعتمد النظام على مفهوم Load Estimation، أي تقدير الحمل الحالي على كل خادم.

آلية العمل:

يقوم النظام بمراقبة مجموعة من المؤشرات لكل خادم، مثل:

- عدد الطلبات الحالية .
- العمليات النشطة .
- زمن الاستجابة المتوقع .
- مستوى الضغط الحالي .

ثم يتم حساب قيمة تقديرية للحمل (Estimated Load) لكل خادم، وبعد ذلك يتم اختيار الخادم الأقل ضغطاً وإرسال الطلب إليه للحصول على أفضل أداء.

-3 Hybrid Load Balancing

تم استخدام النظام الهجين بهدف الاستفادة من مزايا الطريقتين معاً، ومن خلال الدمج بينهما تم الحصول على توزيع متوازن، واستجابة أسرع.

كيفية تطبيق النظام داخل المشروع

تم إنشاء مجموعة من الخوادم الوهمية (Simulated Servers) ، وكل خادم يمتلك:

- معرف خاص .
- حالة تشغيل .
- قيمة حمل حالية .
- عدد الطلبات النشطة .

وعند وصول طلب جديد يتم تحديد الخادم التالي باستخدام Round Robin وفحص حالة الأحمال الحالية، فإذا كان الخادم المحدد مزدحماً يتم تحويل الطلب إلى خادم أقل ضغطاً، ويتم تحديث بيانات الأحمال بعد تنفيذ الطلب.

GET /api/load-balance

HTTP 200 OK

Allow: GET, OPTIONS

Content-Type: application/json

Vary: Accept

```
{
  "message": "Request distributed successfully",
  "round_robin": "Server-3",
  "smart_choice": "Server-2",
  "servers_status": {
    "Server-1": {
      "requests": 3,
      "load": 58
    },
    "Server-2": {
      "requests": 3,
      "load": 50
    },
    "Server-3": {
      "requests": 3,
      "load": 53
    }
  }
}
```

GET /api/load-balance

HTTP 200 OK

Allow: GET, OPTIONS

Content-Type: application/json

Vary: Accept

```
{
  "message": "Request distributed successfully",
  "round_robin": "Server-2",
  "smart_choice": "Server-1",
  "servers_status": {
    "Server-1": {
      "requests": 3,
      "load": 55
    },
    "Server-2": {
      "requests": 3,
      "load": 93
    },
    "Server-3": {
      "requests": 2,
      "load": 84
    }
  }
}
```

بعض التحسينات على الطلب الخامس ومقارنة نتائج

يُعطى المتطلب على مستويين:

1. محاكاة منطقية داخل التطبيق

- الملف: store/services/load_balancer.py
- نقطة النهاية: GET /api/load-balance
- يُقارن بين Round Robin (تتأوب دوري) واختيار الخادم الأقل حملاً (Smart Choice).
- الغرض: توضيح الفرق بين الاستراتيجيتين دون توجيه HTTP فعلي.

2. توزيع حقيقي عبر Docker

الاستراتيجية	المنفذ	البنية	فكرة العمل	ملف المشروع الموجود ضمنه
Least Response Time (LRT)	8080	lb	يختار الخادم ذا أقل متوسط زمن استجابة (نافذة 20 طلباً).	load_balancer/lrt_proxy.py
Round Robin	8081	lb-rr	توزيع بالتناوب على web1, web2, web3.	load_balancer/rr_proxy.py
بدون موزّع (Baseline)	8001	web1 (مباشر)	-	-

البنية العامة:

- ثلاث حاويات Django متطابقة
- MariaDB
- Redis

منهجية المقارنة

- الأداة: Locust (قراءة فقط)
- السيناريو: 50 مستخدم، مدة 60 ثانية
- المسارات: /api/product/all, /api/product/best-sellers/
- الأمر: python stress_test/run.py lb

النتائج

المؤشر	LRT (8080)	(8081) Round-Robin	مباشر web1 (8001)
إجمالي الطلبات	3,285	4,617	4,234
RPS	57.28	79.03	72.57
متوسط الاستجابة (ms)	633.65	403.62	460.68
p95 (ms)	1,100	940	740
نسبة الفشل	%0	%0	%0
استقرار النظام	مستقر	مستقر	مستقر

التحليل والتبرير

1. Round-Robin

حقق أعلى (RPS 79.03) throughput وأقل متوسط استجابة (403 ms) في هذا السenario، لأن الخوادم الثلاث متطابقة تقنياً، والاختبار قراءة فقط دون اختناق موضع على خادم واحد.

2. LRT

أظهر أداء أقل (RPS 57.28) بسبب تكلفة تتبع أزمدة الاستجابة واتخاذ قرار التوجيه. ميزته تظهر عندما تختلف قدرات الخوادم أو يكون الحمل غير متوازن؛ عندها يوجه الطلبات نحو الأسرع استجابة.

3. الاتصال المباشر

أعطى أفضل p95 (740 ms) لغياب طبقة ال-proxy، لكنه لا يحقق التوزيع على عدة backends.

المتطلب السادس Distributed Caching

قبل تطبيق الحل

في الأنظمة التي تحتوي على أكثر من خادم (Multiple Application Servers)، يتم توزيع طلبات المستخدمين على عدة خوادم بهدف زيادة قابلية التوسع وتحسين الأداء. ولكن عند استخدام Cache محلي داخل كل خادم (Local Memory Cache)، تظهر عدة مشاكل، منها:

- كل خادم يحتفظ بنسخة مستقلة من البيانات.
 - عند تحديث البيانات في أحد الخوادم تبقى النسخ الموجودة في بقية الخوادم قديمة (Stale Data).
 - في حال انتقال المستخدم إلى خادم آخر بسبب ال-Load Balancer فإن البيانات المخزنة في الكاش لن تكون موجودة، مما يؤدي إلى إعادة قراءة البيانات من قاعدة البيانات.
 - تزداد الأحمال على قاعدة البيانات نتيجة تكرار عمليات القراءة نفسها من أكثر من خادم.
- لذلك فإن استخدام Local Cache لا يعتبر مناسباً في الأنظمة الموزعة (Distributed Systems).

الهدف من الحل

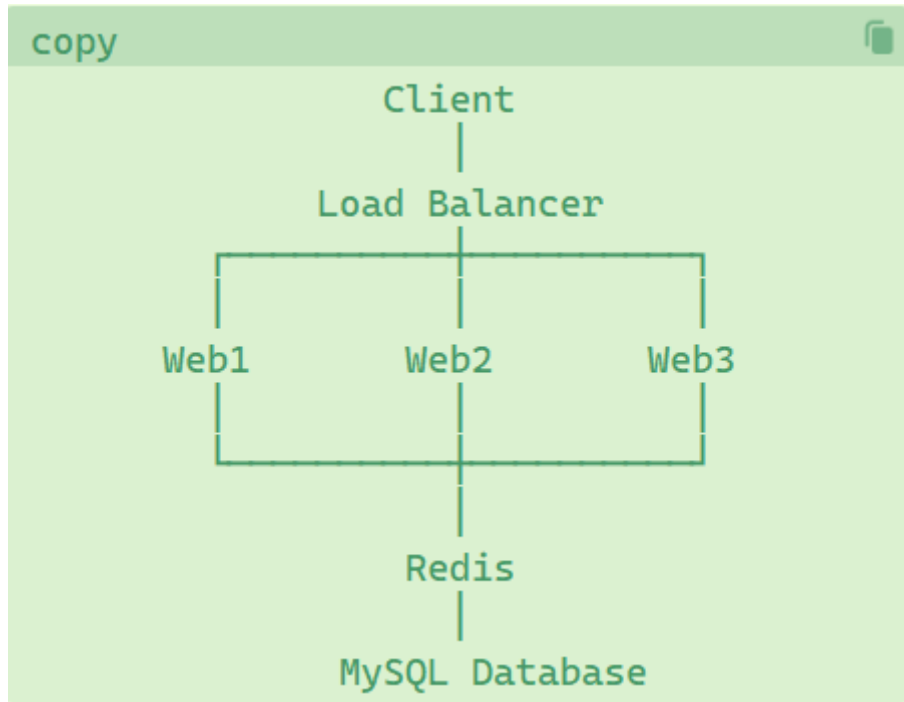
يهدف هذا المتطلب إلى تحقيق ما يلي:

- إنشاء Distributed Cache مشترك بين جميع خوادم التطبيق.
- تقليل عدد عمليات القراءة من قاعدة البيانات.
- تقليل زمن الاستجابة (Response Time).
- الحفاظ على اتساق البيانات (Consistency) بين جميع الخوادم.
- تحسين قابلية التوسع (Scalability).

الحل المعتمد

تم اعتماد Redis كـ Distributed Cache Server بحيث لا يتم تخزين البيانات داخل كل Application Server، وإنما يتم تخزينها داخل خادم Redis مستقل، وتتصل جميع خوادم التطبيق بنفس الـ Redis.

البنية المستخدمة أصبحت كالتالي:



بهذه الطريقة أصبحت جميع الخوادم تشترك بنفس الكاش.

آلية العمل

تم تطبيق أسلوب Cache Aside Pattern داخل النظام، وكانت آلية العمل كما يلي:

1. يرسل المستخدم طلباً للحصول على بيانات المنتج.
2. يقوم الخادم أولاً بالبحث عن البيانات داخل Redis.
3. في حال وجود البيانات (Cache Hit) يتم إرجاعها مباشرة دون الوصول إلى قاعدة البيانات.
4. في حال عدم وجود البيانات (Cache Miss) يتم جلبها من قاعدة البيانات.
5. بعد قراءة البيانات يتم تخزينها داخل Redis لإعادة استخدامها في الطلبات القادمة.
6. عند تعديل المنتج أو حذفه أو تغيير كميته يتم حذف نسخة الكاش الخاصة به (Cache Invalidation) لضمان عدم إرجاع بيانات قديمة.

تطبيق Distributed Cache داخل المشروع

تم استخدام Redis في أكثر من سيناريو داخل المشروع، منها:

- تخزين بيانات المنتج عند طلبه لأول مرة.
- حذف الكاش عند تعديل المنتج أو حذفه أو تغيير كميته.
- تخزين قائمة أكثر المنتجات مبيعاً (Best Sellers).
- استخدام Redis Lock لمنع أكثر من خادم من إعادة بناء نفس الكاش في الوقت نفسه (Cache Stampede Protection).

كما تم استخدام مفهوم Fresh Cache و Stale Cache بحيث:

- يتم إرجاع البيانات الحديثة (Fresh Cache) طالما أنها صالحة.
- عند انتهاء صلاحية الـ Fresh Cache يسمح لخادم واحد فقط بإعادة بنائها.
- في أثناء إعادة البناء تقوم بقية الخوادم بإرجاع نسخة Stale Cache بدلاً من الضغط على قاعدة البيانات.
- في حال عدم وجود أي نسخة يتم الرجوع إلى قاعدة البيانات.

لماذا تم اختيار Redis؟

تمت دراسة خيارين رئيسيين:

Memcached	Redis
Key-Value فقط.	يدعم أنواع بيانات متعددة (Lists, Sets, Hashes).
يعتمد على الذاكرة فقط.	يدعم Persistence.
أبسط لكنه أقل مرونة.	يدعم Distributed Caching بسهولة.
لا يدعم آليات Lock بشكل مدمج.	يدعم Distributed Lock.
مناسب للكاش البسيط فقط.	مناسب لتطبيقات التجارة الإلكترونية.

لذلك تم اختيار Redis لأنه يحقق متطلبات المشروع، ويدعم كل من Distributed Cache وDistributed Lock باستخدام خادم واحد.

نتائج تطبيق الحل

بعد تطبيق Distributed Cache أصبحت جميع خوادم التطبيق تستخدم نفس البيانات المخزنة داخل Redis، مما أدى إلى:

- تقليل عدد الاستعلامات المرسلة إلى قاعدة البيانات.
- تقليل زمن الاستجابة للطلبات المتكررة.
- مشاركة الكاش بين جميع Application Servers.
- منع إعادة بناء الكاش من أكثر من خادم في الوقت نفسه.
- تحسين قابلية التوسع عند إضافة خوادم جديدة.
- تقليل الضغط على قاعدة البيانات مع زيادة عدد المستخدمين.

اختبار التنفيذ

تم تشغيل ثلاثة خوادم للتطبيق خلف Load Balancer مع خادم Redis مشترك، ثم تم إرسال عدة طلبات متزامنة إلى واجهة Best Sellers.

- الحالة الأولى - Cache Miss:

عند إرسال أول طلب، لم تكن البيانات موجودة داخل Redis، لذلك ظهرت الرسائل التالية:

web1 -> CACHE MISS

web1 -> LOCK ACQUIRED

web1 -> REBUILD DONE

ويعني ذلك أن الخادم الأول لم يجد البيانات داخل الكاش، فحصل على Redis Lock ثم أعاد بناء الكاش من قاعدة البيانات وخرن النتائج داخل Redis.

- **الحالة الثانية - Cache Hit :**

عند إرسال الطلبات التالية مباشرة ظهرت الرسالة:

CACHE HIT → FRESH

وهذا يدل على أن البيانات أُعيدت مباشرة من Redis دون تنفيذ أي استعلام على قاعدة البيانات.

- **الحالة الثالثة - انتهاء صلاحية الـ Fresh Cache :**

بعد انتهاء مدة صلاحية الـ Fresh Cache وإرسال عدة طلبات متزامنة، حصل أحد الخوادم فقط على القفل:

web2 -> CACHE MISS

web2 -> LOCK ACQUIRED

بينما ظهرت في بقية الخوادم الرسائل التالية:

web1 -> CACHE MISS

web1 -> LOCK BUSY

web1 -> RETURN STALE

أو

web3 -> CACHE MISS

web3 -> LOCK BUSY

web3 -> RETURN STALE

وهذا يؤكد أن خادماً واحداً فقط قام بإعادة بناء الكاش، بينما قامت بقية الخوادم بإرجاع نسخة **Stale Cache** بدلاً من الضغط على قاعدة البيانات.

قبل تطبيق الحل

في الأنظمة الموزعة التي تعمل بعدة خوادم (Multiple Application Servers)، قد تصل عدة طلبات متزامنة (Concurrent Requests) إلى أكثر من خادم في الوقت نفسه لتنفيذ نفس العملية.

في نظام التجارة الإلكترونية تعتبر عملية Checkout من أكثر العمليات حساسية، لأنها تتضمن:

- التحقق من توفر الكمية في المخزون.
- خصم الكمية من المنتجات.
- خصم الرصيد من حساب المستخدم.
- إنشاء الطلب (Order).

في حال قام أكثر من خادم بتنفيذ هذه العملية للمستخدم نفسه في الوقت نفسه، قد تظهر عدة مشاكل، منها:

- تنفيذ عملية الدفع أكثر من مرة.
- إنشاء أكثر من Order لنفس عملية الشراء.
- خصم كمية المنتجات أكثر من مرة.
- حدوث تعارض (Race Condition) بين الخوادم.
- عدم اتساق البيانات بين الخوادم.

وعلى الرغم من استخدام أقفال قاعدة البيانات (Database Locks) مثل `select_for_update()`، فإن هذه الأقفال تعمل داخل قاعدة البيانات فقط، ولا تمنع أكثر من خادم من بدء تنفيذ نفس العملية في الوقت نفسه.

لذلك أصبح من الضروري وجود آلية تمنع تنفيذ عملية الـ Checkout نفسها من أكثر من خادم قبل الوصول إلى قاعدة البيانات.

الهدف من الحل

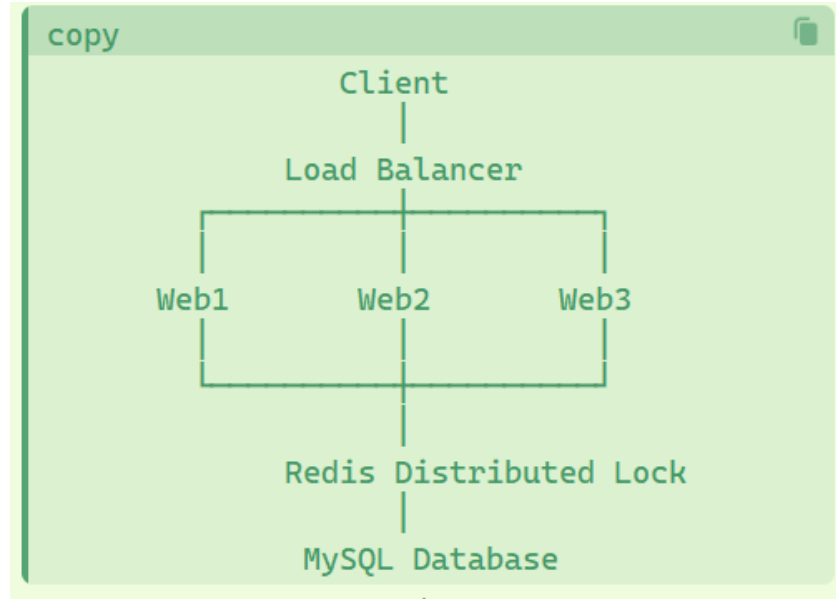
يهدف هذا المتطلب إلى تحقيق ما يلي:

- السماح بتنفيذ عملية Checkout واحدة فقط لكل مستخدم في اللحظة نفسها.
- منع تنفيذ نفس العملية بشكل متزامن على عدة خوادم.
- حماية النظام من مشاكل Race Condition.
- تقليل احتمال إنشاء Orders مكررة.
- الحفاظ على اتساق البيانات (Consistency).
- زيادة موثوقية النظام في البيئات الموزعة.

الحل المعتمد

تم استخدام Redis Distributed Lock ليكون نقطة تنسيق مشتركة بين جميع خوادم التطبيق.

أصبح سير العمل كما يلي:



عند بدء تنفيذ عملية Checkout يحاول الخادم أولاً الحصول على Lock داخل Redis باستخدام معرف المستخدم (checkout:user_id).

إذا نجح بالحصول على الـ Lock يصبح هذا الخادم الوحيد المسموح له بإكمال العملية، بينما يتم رفض جميع الطلبات الأخرى القادمة من بقية الخوادم حتى انتهاء العملية وإطلاق الـ Lock.

آلية العمل

تم تطبيق آلية Distributed Lock بالشكل التالي:

1. يرسل المستخدم طلب تنفيذ عملية Checkout.
 2. يحاول الخادم الحصول على Redis Lock خاص بالمستخدم.
 3. إذا كان الـ Lock متاحاً يتم منحه لهذا الخادم.
 4. يبدأ الخادم بتنفيذ عملية Checkout كاملة.
 5. يتم استخدام select_for_update () داخل قاعدة البيانات لحماية الصفوف أثناء التعديل.
 6. بعد انتهاء العملية يتم تحرير الـ Lock.
 7. إذا وصل طلب آخر أثناء وجود الـ Lock يتم رفضه مباشرة وإرجاع رسالة تفيد بأن عملية Checkout قيد التنفيذ.
- وبذلك لا يسمح إلا لخادم واحد فقط بتنفيذ العملية في أي لحظة.

تطبيق Distributed Lock داخل المشروع

تم استخدام Redis Lock داخل خدمة تنفيذ عملية Checkout (checkout_cart_safely) كما يلي:

- إنشاء Lock باستخدام معرف المستخدم (checkout:user_id).
 - محاولة الحصول على الـ Lock قبل تنفيذ أي عملية.
 - رفض الطلب في حال كان الـ Lock مستخدماً من خادم آخر.
 - تنفيذ جميع خطوات Checkout داخل Transaction واحدة.
 - تحرير الـ Lock في جميع الحالات باستخدام finally لضمان عدم بقاءه محجوزاً.
- كما تم الإبقاء على استخدام select_for_update() داخل قاعدة البيانات لحماية الصفوف أثناء تحديث الكميات والأرصدة. وبذلك أصبح النظام يعتمد على مستويين من الحماية:
- **Distributed Lock** على مستوى جميع خوادم التطبيق.
 - **Database Row Lock** على مستوى قاعدة البيانات.

نتائج تطبيق الحل

بعد تطبيق Distributed Lock أصبحت عملية Checkout أكثر أماناً واستقراراً، حيث تم تحقيق ما يلي:

- منع تنفيذ أكثر من Checkout للمستخدم نفسه في الوقت نفسه.
- منع إنشاء Orders مكررة.
- تقليل احتمال حدوث Race Conditions.
- الحفاظ على اتساق بيانات المخزون والأرصدة.
- توزيع العمل بأمان بين جميع Application Servers.
- زيادة موثوقية النظام عند زيادة عدد المستخدمين والخوادم.

اختبار التنفيذ

للتأكد من صحة تنفيذ Distributed Lock، تم تشغيل أكثر من خادم للتطبيق خلف Load Balancer، ثم إرسال عدة طلبات Checkout متزامنة للمستخدم نفسه.

أظهرت سجلات التنفيذ أن خادماً واحداً فقط استطاع الحصول على Redis Lock وأكمل عملية الشراء، بينما قامت بقية الخوادم برفض الطلبات المتزامنة حتى انتهاء العملية.

كما ظهرت في السجلات رسائل مشابهة لما يلي:

[web2] REDIS LOCK ACQUIRED checkout:43

[web2] REDIS LOCK RELEASED checkout:43

وفي حال وصول طلب متزامن أثناء وجود القفل تظهر رسالة مثل:

REDIS LOCK REJECTED checkout:43

وهذا يؤكد أن Redis Distributed Lock نجح في منع تنفيذ عملية Checkout نفسها من أكثر من خادم في الوقت نفسه، مع الحفاظ على سلامة البيانات ومنع التعارضات.

المتطلب الثامن Transactions

في البداية، نريد الإشارة إلى أن عمل المعاملات يتكامل مع آلية الأقفال (Locks) من أجل ضمان الحصول على نتائج متسقة (Consistent) عند تنفيذ عمليات متزامنة.

تطبيق خصائص ACID

1. Atomicity:

تعني أن جميع العمليات المكونة للمعاملة يجب أن تنجح بالكامل، وإلا يتم التراجع عن جميع التعديلات التي تمت أثناء تنفيذها.

• آلية التحقق:

تم تحقيق ذلك باستخدام Decorator التالي من Django Framework :@transaction.atomic حيث يتم وضعه قبل التابع Function المراد تنفيذ ال transaction عليه.

ونود التنويه إلى أننا استخدمنا @transaction.atomic في بعض المواضع التي لا تحتوي على عمليات مركبة، وذلك لأن استخدامه ضروري عند العمل مع الأقفال التشاؤمية (Pessimistic Locks) المطبقة باستخدام : select_for_update() إذ يحتاج القفل إلى وجود معاملة نشطة (Active Transaction) حتى يبقى فعالاً طوال مدة التنفيذ. أما في حال عدم وجود معاملة، فإن القفل يُحرر مباشرة بعد تنفيذ الاستعلام.

• المواضع التي استخدم فيها @transaction.atomic:

▪ user_create

عند إنشاء مستخدم جديد يتم أيضاً إنشاء حساب بنكي خاص به بالإضافة إلى إنشاء سلة مشتريات فارغة (Cart). وهذه عمليات مترابطة، لذلك لا نريد إنشاء المستخدم إذا فشل إنشاء الحساب البنكي أو السلة، والعكس صحيح.

▪ order_cancel

عند إلغاء الطلب يجب إعادة كميات المنتجات إلى المخزون، بالإضافة إلى إعادة المبلغ المدفوع إلى الحساب البنكي للمستخدم (Refund). ولذلك يجب تنفيذ جميع هذه العمليات ضمن transaction واحدة.

▪ checkout_cart_safely

يُعد هذا التابع المسؤول الرئيسي عن تحويل سلة المشتريات إلى طلب فعلي. وتتضمن العملية:

- التحقق من كفاية الرصيد البنكي.
 - إنشاء الطلب.
 - تحويل عناصر السلة إلى عناصر طلب (Product Orders).
 - تحديث كميات المنتجات في المخزون.
 - اقتطاع قيمة الطلب من الحساب البنكي.
- ولذلك يجب تنفيذ جميع هذه الخطوات ضمن معاملة واحدة.

كما ذكرنا سابقاً، استخدمنا @transaction.atomic في بعض التوابع التي لا تحتوي على عمليات مركبة، وذلك بهدف إبقاء القفل فعالاً طوال فترة تنفيذ الطلب، وهي:

- product_add_quantity
- cart_product_update
- cart_product_create

2. Consistency

يقصد به أن الحالة النهائية للبيانات بعد انتهاء المعاملة يجب ألا تخالف أي قاعدة أو قيد موجود في قاعدة البيانات، مثل السماح بكميات سالبة للمنتجات.

تم تحقيق ذلك من خلال:

- بناء منطق التطبيق بطريقة تمنع الوصول إلى حالات غير صحيحة.
 - استخدام قيود قاعدة البيانات (Database Constraints).
 - الاستفادة من الأقفال لمنع التعديلات المتزامنة غير المرغوبة.
- فعلى سبيل المثال، إذا حاول مستخدم شراء آخر قطعة متوفرة من أحد المنتجات في الوقت نفسه، فإن آلية الأقفال تضمن نجاح عملية شراء واحدة فقط، مما يمنع وصول كمية المنتج إلى قيمة سالبة.

3. Isolation

يقصد بالعزل درجة الفصل بين المعاملات التي تعمل بشكل متزامن.

آلية التحقق:

في تطبيقنا اعتمدنا على مستوى العزل الافتراضي في قاعدة بيانات MySQL ، وهو: Repeatable Read ويُعد هذا المستوى توازناً مناسباً بين الأداء والكفاءة، حيث:

- يمنع قراءة بيانات تم تعديلها داخل معاملة أخرى لم يتم تثبيت تغييراتها (Commit) بعد.

- يمنع مشكلة Non-Repeatable Read حيث يضمن أن المعاملة نفسها ستحصل على نفس البيانات عند قراءتها أكثر من مرة، حتى لو قامت معاملة أخرى بتعديل تلك البيانات وإجراء Commit لها. ويُعتبر هذا المستوى كافياً لمتطلبات التطبيق الحالي.

4. Durability

تعني أنه بعد نجاح المعاملة وإجراء عملية التثبيت (Commit)، يجب أن تبقى التعديلات محفوظة بشكل دائم، وألا تضيع حتى في حال حدوث انقطاع مفاجئ أو عطل في النظام. يتم ضمان هذه الخاصية من قبل قاعدة البيانات نفسها من خلال استخدام تقنية: WAL (Write-Ahead Logging) حيث يتم تسجيل التعديلات أولاً في ملف السجل (Log File) قبل تثبيتها بشكل نهائي داخل قاعدة البيانات. وبالتالي يمكن استعادة البيانات حتى في حال حدوث خطأ مفاجئ. كما تم التأكد من ضبط الإعدادات التالي في MySQL :

```
innodb_flush_log_at_trx_commit = 1
```

وهذا يضمن كتابة كل عملية Commit مباشرة على القرص (Disk)، وليس فقط في الذاكرة (Memory)، مما يعزز من موثوقية حفظ البيانات.

الاختبارات والنتائج

قاعدة البيانات الأساسية لدينا تملك 300 مستخدم و300 منتج، وملفات الاختبار تقوم بإنشاء مستخدمين ومنتجات مع المعلومات المطلوبة بما يتوافق مع الاختبار

الاختبار الأول: التحقق من عمل ال transactions بشكل صحيح

ملف الاختبار: store/tests/demo_transactions.py

يحتوي الملف على أربعة اختبارات:

1. فشل إنشاء مستخدم بسبب فشل إنشاء الحساب البنكي.
2. فشل عملية الدفع بعد إنشاء الطلب.
3. فشل إتمام الطلب بسبب فشل إنشاء ProductOrder.
4. فشل إلغاء الطلب مع التأكد من إعادة الكميات والمبلغ المالي إلى الحساب البنكي.

تم تنفيذ الاختبار مع استخدام Transactions وبدونها.

• النتيجة مع استخدام المعاملات

تم التراجع عن جميع التعديلات (Rollback) عند حدوث أي خطأ، وعادت قاعدة البيانات إلى حالتها السابقة بشكل كامل.

يوجد نتيجة التنفيذ كتقرير في الملف: store\tests\demo_results_with_transaction.txt وهذا توضيح نجاح العمليات من قاعدة البيانات:

```
mysql> select id, username from auth_user order by id desc limit 10;
+-----+-----+
| id | username |
+-----+-----+
| 304 | demo_buyer3 |
| 303 | demo_buyer2 |
| 302 | demo_buyer |
| 300 | user299 |
| 299 | user298 |
| 298 | user297 |
| 297 | user296 |
| 296 | user295 |
| 295 | user294 |
| 294 | user293 |
+-----+-----+
10 rows in set (0.02 sec)

mysql> select * from store_cart order by id desc limit 10;
+-----+-----+
| id | user_id |
+-----+-----+
| 304 | 304 |
| 303 | 303 |
| 302 | 302 |
+-----+-----+
3 rows in set (0.00 sec)

mysql> select * from store_bank order by id desc limit 10;
+-----+-----+-----+
| id | balance | user_id |
+-----+-----+-----+
| 303 | 800.00 | 304 |
| 302 | 1000.00 | 303 |
| 301 | 1000.00 | 302 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

نسبة للاختبار الأول نلاحظ أن المستخدم ذو الرقم 301 لم يتم إنشاؤه ولا يملك سلة او حساب بنكي, بالتالي تم التراجع عن إنشائه بنجاح.

```
mysql> select id, username from auth_user order by id desc limit 5;
+-----+-----+
| id | username |
+-----+-----+
| 304 | demo_buyer3 |
| 303 | demo_buyer2 |
| 302 | demo_buyer |
| 300 | user299 |
| 299 | user298 |
+-----+-----+
5 rows in set (0.00 sec)

mysql> select * from store_bank order by id desc limit 5;
+-----+-----+-----+
| id | balance | user_id |
+-----+-----+-----+
| 303 | 800.00 | 304 |
| 302 | 1000.00 | 303 |
| 301 | 1000.00 | 302 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> select * from store_cart order by id desc limit 5;
+-----+-----+
| id | user_id |
+-----+-----+
| 304 | 304 |
| 303 | 303 |
| 302 | 302 |
+-----+-----+
3 rows in set (0.00 sec)

mysql> select * from store_order order by id desc limit 5;
+-----+-----+-----+-----+-----+
| id | status | total_amount | created_at | user_id |
+-----+-----+-----+-----+-----+
| 302 | pending | 200.00 | 2026-06-20 15:24:02.167717 | 304 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

نسبة للاختبار الثاني والثالث كلاهما تم فشل إنشاء طلب بسبب حدوث خطأ ما، ونلاحظ أن المستخدمان 302 و 303 يملكان السلة لكن لا يملكان أي طلبات، كما أن حسابهم البنكي مازال يملك نفس المبلغ المالي (تم إنشاء حسابهما بمبلغ 1000) بالتالي أيضا تم التراجع بشكل صحيح.

ونلاحظ أيضا نسبة للاختبار الرابع أن عملية الإلغاء فشلت ومازال الطلب pending ولم يصبح cancelled والحساب البنكي مازال يملك نفس المبلغ (تم إنشاءه بالمبلغ 800).

- النتيجة بدون استخدام المعاملات

بقيت بعض البيانات محفوظة رغم فشل العملية بالكامل، مما أدى إلى ظهور حالة غير متسقة في قاعدة البيانات، وهو ما يوضح أهمية استخدام المعاملات.

يوجد نتيجة التنفيذ كتقرير في الملف: store\tests\demo_results_without_transaction.txt وهذا توضيح نجاح العمليات من قاعدة البيانات:

```
mysql> select id, username from auth_user order by id desc limit 5;
+-----+-----+
| id | username |
+-----+-----+
| 304 | demo_buyer3 |
| 303 | demo_buyer2 |
| 302 | demo_buyer |
| 301 | demo_user |
| 300 | user299 |
+-----+-----+
5 rows in set (0.03 sec)

mysql> select * from store_bank order by id desc limit 5;
+-----+-----+-----+
| id | balance | user_id |
+-----+-----+-----+
| 303 | 1000.00 | 304 |
| 302 | 800.00 | 303 |
| 301 | 1000.00 | 302 |
+-----+-----+-----+
3 rows in set (0.01 sec)

mysql> select * from store_cart order by id desc limit 5;
+-----+-----+
| id | user_id |
+-----+-----+
| 304 | 304 |
| 303 | 303 |
| 302 | 302 |
| 301 | 301 |
+-----+-----+
4 rows in set (0.00 sec)
```

تم إنشاء حساب للمستخدم 301 مع سلة مشتريات لكن لم يتم إنشاء حساب بنكي.

```
mysql> select id, username from auth_user order by id desc limit 5;
+-----+-----+
| id | username |
+-----+-----+
| 304 | demo_buyer3 |
| 303 | demo_buyer2 |
| 302 | demo_buyer |
| 301 | demo_user |
| 300 | user299 |
+-----+-----+
5 rows in set (0.00 sec)

mysql> select * from store_bank order by id desc limit 3;
+-----+-----+-----+
| id | balance | user_id |
+-----+-----+-----+
| 303 | 1000.00 | 304 |
| 302 | 800.00 | 303 |
| 301 | 1000.00 | 302 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> select * from store_order order by id desc limit 5;
+-----+-----+-----+-----+-----+
| id | status | total_amount | created_at | user_id |
+-----+-----+-----+-----+-----+
| 302 | pending | 200.00 | 2026-06-20 16:31:46.264150 | 304 |
| 301 | pending | 200.00 | 2026-06-20 16:31:44.948884 | 303 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> select * from store_productorder limit 5;
+-----+-----+-----+-----+-----+
| id | quantity | price | order_id | product_id |
+-----+-----+-----+-----+-----+
| 1938 | 2 | 100.00 | 302 | 303 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

رغم فشل إنشاء ال productorder لطلب لمستخدم 303, إلا أن الطلب تم إنشاءه فارغاً وتم اقتطاع المبلغ من حسابه البنكي (رصيده الأساسي 1000)

ومن نفس الصورة, نلاحظ بالنسبة للمستخدم 304 الذي طلب إلغاء الطلب, لكن حدث خطأ عند إعادة الكميات من المنتج للمخزون, بالتالي فشلت عملية الإلغاء (حالة الطلب pending) ولم يتم إعادة كمية المنتج للمخزون حيث كانت بالأساس 8

```
mysql> select * from store_product order by id desc limit 10;
+-----+-----+-----+-----+
| id | name | price | quantity |
+-----+-----+-----+-----+
| 303 | Demo Product 3 | 100.00 | 8 |
```

لكن مع ذلك تم إعادة المبلغ لحساب المستخدم (رصيده الأساسي 800 وتم إعادة سعر الطلب وهو 200)

الاختبار الثاني: التحقق من عمل المعاملات في بيئة متزامنة

ملف الاختبار: demo_concurrency_trans.py

تم اختبار الحالات التالية:

1. قيام مستخدمين بطلب آخر قطعة من منتج في الوقت نفسه.
2. قيام مديرين بتعديل المنتج نفسه بشكل متزامن.
3. إلغاء طلب وتأكيده طلب آخر في الوقت نفسه للمستخدم نفسه.
4. قيام المدير بإضافة كمية جديدة إلى المخزون بالتزامن مع قيام مستخدم بتأكيد طلب شراء.

يوجد نتيجة التنفيذ كتقرير في الملف: store\tests\demo_concurrency_results_with_transaction.txt

وهذا توضيح نجاح العمليات من قاعدة البيانات:

```
mysql> select id, username from auth_user order by id desc limit 10;
+----+-----+
| id | username |
+----+-----+
| 318 | concurrent_restock_admin |
| 317 | concurrent_restock_buyer |
| 316 | concurrent_mixed |
| 315 | concurrent_admin2 |
| 314 | concurrent_admin |
| 313 | concurrent_buyer2 |
| 312 | concurrent_buyer1 |
| 304 | demo_buyer3 |
| 303 | demo_buyer2 |
| 302 | demo_buyer |
+----+-----+
10 rows in set (0.00 sec)

mysql> select * from store_order order by id desc limit 5;
+----+-----+-----+-----+-----+
| id | status | total_amount | created_at | user_id |
+----+-----+-----+-----+-----+
| 308 | pending | 100.00 | 2026-06-20 15:41:19.937052 | 317 |
| 307 | pending | 200.00 | 2026-06-20 15:41:15.172619 | 316 |
| 306 | cancelled | 100.00 | 2026-06-20 15:41:13.086072 | 316 |
| 305 | pending | 50.00 | 2026-06-20 15:41:08.862936 | 312 |
+----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> select * from store_bank order by id desc limit 5;
+----+-----+-----+
| id | balance | user_id |
+----+-----+-----+
| 311 | 900.00 | 317 |
| 310 | 400.00 | 316 |
| 309 | 1000.00 | 313 |
| 308 | 950.00 | 312 |
+----+-----+-----+
4 rows in set (0.00 sec)
```

الاختبار الأول: لدينا المستخدمين concurrent_buyer1 و concurrent_buyer2, نلاحظ أنه تم إنشاء الطلب فقط للمستخدم 312, وتم اقتطاع المبلغ من حسابه البنكي أما المستخدم 313 لم تنجح عملية الطلب بالنسبة له.

ومن هذه الصورة أيضا يتضح نتيجة الاختبار الثالث, حيث قام المستخدم 316 بإلغاء الطلب 306 (حالته cancelled) وقام بتأكيد طلبية أخرى 307 (الحالة pending) وتم تحديث حسابه البنكي بالشكل الصحيح (القيمة في حسابه البنكي بعد الطلبية الأولى 500)

```
mysql> select * from store_product order by id desc limit 10;
+-----+-----+-----+-----+
| id | name | price | quantity |
+-----+-----+-----+-----+
| 313 | Concurrent Restock | 10.00 | 55 |
| 312 | Concurrent Prod2 | 200.00 | 4 |
| 311 | Concurrent Prod1 | 100.00 | 11 |
| 310 | Concurrent product AdminA | 100.00 | 100 |
| 309 | Concurrent Hot Item | 50.00 | 0 |
| 303 | Demo Product 3 | 100.00 | 8 |
| 302 | Demo Product 2 | 100.00 | 10 |
| 301 | Demo Product | 100.00 | 10 |
| 300 | Anker Memory Foam Pillow Elite | 41.08 | 2774 |
| 299 | Anker Non-Stick Cookware Set Pro | 408.54 | 1268 |
+-----+-----+-----+-----+
10 rows in set (0.00 sec)
```

الاختبار الثاني: توضح لدينا أن عمليتنا التعديل على كمية المنتج كانت ناجحة، ويوضح لنا اسم المنتج 310 أن المدير الأول هو آخر من قام بالتعديلات (قمنا بالاختبار بالتعديل على اسم المنتج لنستطيع التمييز أسرع من قام بالتعديل أخيرا)

```
mysql> select * from store_product order by id desc limit 10;
+-----+-----+-----+-----+
| id | name | price | quantity |
+-----+-----+-----+-----+
| 313 | Concurrent Restock | 10.00 | 55 |
| 312 | Concurrent Prod2 | 200.00 | 4 |
| 311 | Concurrent Prod1 | 100.00 | 11 |
| 310 | Concurrent product AdminA | 100.00 | 100 |
| 309 | Concurrent Hot Item | 50.00 | 0 |
| 303 | Demo Product 3 | 100.00 | 8 |
| 302 | Demo Product 2 | 100.00 | 10 |
| 301 | Demo Product | 100.00 | 10 |
| 300 | Anker Memory Foam Pillow Elite | 41.08 | 2774 |
| 299 | Anker Non-Stick Cookware Set Pro | 408.54 | 1268 |
+-----+-----+-----+-----+
10 rows in set (0.00 sec)

mysql> select * from store_order order by id desc limit 5;
+-----+-----+-----+-----+-----+
| id | status | total_amount | created_at | user_id |
+-----+-----+-----+-----+-----+
| 308 | pending | 100.00 | 2026-06-20 15:41:19.937052 | 317 |
| 307 | pending | 200.00 | 2026-06-20 15:41:15.172619 | 316 |
| 306 | cancelled | 100.00 | 2026-06-20 15:41:13.086072 | 316 |
| 305 | pending | 50.00 | 2026-06-20 15:41:08.862936 | 312 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> select * from store_productorder limit 5;
+-----+-----+-----+-----+-----+
| id | quantity | price | order_id | product_id |
+-----+-----+-----+-----+-----+
| 1922 | 1 | 50.00 | 305 | 309 |
| 1923 | 1 | 100.00 | 306 | 311 |
| 1924 | 1 | 200.00 | 307 | 312 |
| 1925 | 10 | 10.00 | 308 | 313 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

بالنسبة للاختبار الأخير، عدد المخزون من المنتج 313 كان 15، تم إضافة 50 من قبل الادمن وعند تأكيد المستخدم 317 للطلبية الخاصة به التي تخوي على كمية 10 من المنتج نفسه، اصبحت الكمية النهائية للمنتج 55.

التنفيذ

• أدوات القياس

Locust: locust/locustfile.py -

Pipeline: stress_test/run.py (أمر stress) -

فحص سلامة البيانات: store/management/commands/stress_check.py -

• سيناريو الضغط

- 100 مستخدم متزامن عبر مورّع LRT (8080):

- عمليات: تسجيل دخول، checkout، تأكيد دفع، تصفح، مسار طلب كامل

- الحماية: select_for_update داخل transaction.atomic في store/services/inventory.py

• معايير القبول

آلية التحقق	المعيار
GET /api/product/all — health() بعد الاختبار	عدم انهيار النظام
معادلة المخزون و عدد المنتجات ذات الكمية السالبة	عدم فقدان البيانات
Locust: Request Count و Failure Count	استمرار الخدمة

النتائج

المؤشر	القيمة
المستخدمون المتزامنون	100
إجمالي الطلبات	825
نسبة الفشل	22.18%
RPS	4.65
p95 (ms)	40,000
النظام بعد الاختبار	يعمل
مخزون سالب	0
سلامة المخزون	متحققة
معادلة التحقق	$2396 + 737564 = 739960$
Deadlocks	0

0	Constraint violations
PASSED	الحكم النهائي

التحليل

1. نسبة الفشل 22.18% تعكس رفض طلبات (نفاد مخزون، timeout، أخطاء HTTP 4xx/5xx)، وليست انهيار للنظام. Locust يصنّف هذه ك failures دون توقف الخدمة
2. p95 = 40,000 ms مرتفع ومتوقع في enario checkout تحت 100 مستخدم، بسبب:
 - قفل الصفوف (select_for_update)
 - محاكاة بوابة الدفع
 - تنافس متزامن على موارد قاعدة البيانات
3. سلامة البيانات: هي المعادلة $739960 = 737564 + 2396$ تثبت أن إجمالي المخزون الابتدائي يساوي المخزون النهائي زائد الوحدات المباعة أثناء الاختبار. لا توجد منتجات بكمية سالبة.
4. مقارنة مع تجربة Race (بدون قفل): في race_experiment سجّل 19 deadlock و 21 constraint violation مع RACE_DETECTED. تحت الضغط مع select_for_update، النتيجة PASSED — ما يؤكد فعالية آلية الحماية.

الاستنتاج: تحقق شرط الطلب 9: النظام بقي يعمل والبيانات متسقة ولم يحدث تلف في المخزون

يوجد نتائج شاملة في تقرير الطلب العاشر

المتطلب العاشر Benchmarking & Bottleneck Analysis

التنفيذ

أولا : Prometheus

المكوّن	الملف	الوظيفة
django-prometheus	project/settings.py, project/urls.py	مقاييس HTTP على /metrics
SystemMetricsMiddleware	store/system_metricsmiddleware.py	تحديث المقاييس كل طلب
Gauges	store/metrics.py	CPU ، RAM ، عدد Threads

ثانيا : Redis Cache

الملف: store/views/product_view.py

الاستراتيجية	Endpoint
Cache-Aside — مفتاح {id} product:	GET /api/product/show/<id>
Fresh + Stale + Redis Lock + DB Fallback	GET /api/product/best-sellers

ثالثا : إدارة الموارد (Waitress)

- الملف: run_server.py

- $\text{threads} = \text{cpu_cores} \times 2$

- $\text{connection_limit} = \text{threads} \times 50$

النتائج والمقارنات الرقمية النهائية

تقرير مقارنة اختبارات الأداء والتزام									
PP E-commerce Project — Non-Functional Requirements (5–10) آخر تحديث: 21T04:04:07.119800+00:00-06-2026									
جدول المقارنة الشامل									
النتيجة	RPS	p95 ms	متوسط ms	نسبة الفشل %	الطلبات	المستخدمون	نوع القفل	موزع الحمل	التجربة
PASSED	4.65	40,000	19,204.6	22.18	825	100	select_for_update	lrt	اختبار الضغط — 100 مستخدم (الطلب 9)
RACE DETECTED	—	—	—	—	50	5	none	—	تجربة Race — بدون select_for_update (الطلب 7)
OK	57.28	1,100	633.65	0	3285	50	—	lrt	موزع 8080 — LRT
OK	79.03	940	403.62	0	4617	50	—	round_robin	Round-Robin — 8081
OK	72.57	740	460.68	0	4234	50	—	none	مباشر 8001 — web1

OK	72.57	740	460.68	0	4234	50	—	none	مباشر 8001 — web1
----	-------	-----	--------	---	------	----	---	------	-------------------

الطلب 9 — اختبار الاستقرار تحت الضغط



الطلب 7 — مع القفل vs بدون القفل

الطلب 7 — مع القفل vs بدون القفل

بدون select_for_update	مع select_for_update	
19	0	Deadlocks
21	0	انتهاكات القيد
نعم ✓	لا X	Race detected
0	0	مخزون سالب
RACE DETECTED	PASSED	النتيجة

الطلب 5 — مقارنة مؤّعات الحمل

